SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.* (BL2, BL3)

Activity Tool : Case Study (Medium)
Assessment Tool Weightage: 30%

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	**5**
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	**5**
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	**5**
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	**5**
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	**5**
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	**5**
7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	**5**
8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	**5**
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	**5**
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	**5**

Code of Conduct:

*I **M.Vishnu Vardhan Reddy** (192411053) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ***M.Vishnu Vardhan Reddy***

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q1. Online Shopping Platform – Normalization to 3NF

Given Relation

Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

An online shopping platform stores customer, order, and product details in a single relation.

1. Functional Dependencies

A functional dependency shows that one attribute determines another attribute. It is written as **A → B**.

The functional dependencies are:

1. **OrderID → CustID**
   Each order belongs to one customer.
2. **CustID → CustName**
   Each customer ID identifies one customer name.
3. **ProdID → ProdName, Price**
   Each product ID determines its product name and price.
4. **(OrderID, ProdID) → Qty**
   The quantity depends on the particular product in a particular order.

Therefore, the **candidate key is (OrderID, ProdID)**.

2. First Normal Form (1NF)

A relation is in 1NF when all attributes contain **atomic (single) values** and there are no repeating groups.

The relation is in **1NF** because:

- All attributes contain atomic values.
- There are no repeating groups.
- Each row represents an order-product combination.

3. Second Normal Form (2NF)

A relation is in 2NF when it is in **1NF** and every non-key attribute is **fully dependent on the entire primary key**, with no partial dependency.

The candidate key is **(OrderID, ProdID)**.

There are partial dependencies:

- OrderID → CustID
- CustID → CustName
- ProdID → ProdName, Price

To remove these partial dependencies, decompose the relation into:

Customer(CustID, CustName)

Product(ProdID, ProdName, Price)

Orders(OrderID, CustID)

OrderItem(OrderID, ProdID, Qty)

4. **Third Normal Form (3NF)**
   A relation is in 3NF when it is in **2NF** and there is **no transitive dependency** between non-key attributes.

Now check for transitive dependencies:

- In **Customer**, CustID → CustName
- In **Product**, ProdID → ProdName, Price
- In **Orders**, OrderID → CustID
- In **OrderItem**, (OrderID, ProdID) → Qty

There is no non-key attribute determining another non-key attribute. Therefore, all the decomposed relations are in **3NF**.

Final 3NF Schema

Relation	Attributes
Customer	CustID (PK), CustName
Product	ProdID (PK), ProdName, Price
Orders	OrderID (PK), CustID (FK)
OrderItem	OrderID (PK, FK), ProdID (PK, FK), Qty

Conclusion

The original Order relation is normalized up to **3NF** by separating customer, product, order, and order-item information. This reduces **data redundancy** and prevents **insertion, deletion, and update anomalies** while maintaining data integrity.

Q2. Hospital Management System – Normalize to BCNF

1. Given Relation

Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

The hospital stores patient details, doctor details, and ward details in one table. This may result in repeated information and data anomalies.

2. Functional Dependency (FD)

A **Functional Dependency** means that one attribute uniquely determines another attribute.

The FDs are:

- PID → PName, DocID, WardNo
- DocID → DocName, Specialization
- WardNo → WardName

Candidate Key: PID

3. Problems in the Relation

Update Anomaly:
If a doctor's name or specialization changes, the same information must be updated for every patient of that doctor.

Insertion Anomaly:
A new doctor or ward cannot be stored independently without assigning a patient.

Deletion Anomaly:
If the last patient assigned to a particular doctor is deleted, the doctor's information may also be lost.

4. 1NF

The relation satisfies **1NF** because:

- Each field contains a single atomic value.
- There are no repeating groups.
- Each record represents one patient.

5. 2NF

The candidate key is PID, which is a **single attribute**. Therefore, partial dependency cannot occur.

Hence, the relation is in **2NF**.

6. 3NF

There are transitive dependencies:

- PID → DocID → DocName, Specialization
- PID → WardNo → WardName

Therefore, doctor and ward information should be separated.

7. BCNF Decomposition

The relation is decomposed into:

Patient Table

Patient(PID, PName, DocID, WardNo)

- PID → Primary Key
- DocID → Foreign Key
- WardNo → Foreign Key

Doctor Table

Doctor(DocID, DocName, Specialization)

- DocID → Primary Key

Ward Table

Ward(WardNo, WardName)

- WardNo → Primary Key

8. BCNF Verification

For **Patient**:

PID → PName, DocID, WardNo

Here, PID is the candidate key.

For **Doctor**:

DocID → DocName, Specialization

Here, DocID is the candidate key.

For **Ward**:

WardNo → WardName

Here, WardNo is the candidate key.

Therefore, every determinant is a candidate key in its respective relation. Hence, the relations satisfy **BCNF**.

9. Final Normalized Schema

Relation	Attributes
Patient	PID (PK), PName, DocID (FK), WardNo (FK)
Doctor	DocID (PK), DocName, Specialization
Ward	WardNo (PK), WardName

Conclusion

The original **Patient** relation is successfully normalized up to **BCNF**. The decomposition separates patient, doctor, and ward information, thereby reducing redundancy and eliminating **update, insertion, and deletion anomalies**. It also maintains data integrity using primary and foreign keys.

Q3. University Enrollment System – Apply 2NF

1. Given Relation

Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade)

The university stores student and course information along with the student's grade in a single relation.

2. Functional Dependencies

A **Functional Dependency (FD)** means one attribute determines another attribute.

The FDs are:

- SID → SName
- CourseID → CourseName, Faculty
- (SID, CourseID) → Grade

Candidate Key: (SID, CourseID)

3. Problems in the Relation

Update Anomaly:
If a course name or faculty changes, it must be updated in multiple enrollment records.

Insertion Anomaly:
A new course cannot be stored unless at least one student is enrolled.

Deletion Anomaly:
If the last student enrolled in a course is deleted, the course information may also be lost.

4. 1NF

The relation is in **1NF** because all attributes contain atomic values and there are no repeating groups.

5. 2NF

The candidate key is (SID, CourseID).

There are partial dependencies:

- SID → SName
- CourseID → CourseName, Faculty

To remove these partial dependencies, decompose the relation.

6. 2NF Decomposition

Student

Student(SID, SName)

Course

Course(CourseID, CourseName, Faculty)

Enrollment

Enrollment(SID, CourseID, Grade)

Here:

- SID is the primary key of Student.
- CourseID is the primary key of Course.
- (SID, CourseID) is the composite primary key of Enrollment.

Final 2NF Schema

Relation	Attributes
Student	SID (PK), SName
Course	CourseID (PK), CourseName, Faculty
Enrollment	SID (PK/FK), CourseID (PK/FK), Grade

Conclusion

The original **Enrollment** relation is successfully decomposed into **Student, Course, and Enrollment** tables in **2NF**. This removes partial dependencies, reduces data redundancy, and prevents insertion, deletion, and update anomalies.

Q4. Airline Booking System – Normalize up to BCNF

1. Given Relation

Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

The airline stores passenger, flight, seat, and journey details in one relation.

2. Functional Dependencies

- PassID → PassName
- FlightNo → DeptCity, ArrCity
- (FlightNo, Date, PassID) → Seat

Candidate Key: (FlightNo, Date, PassID)

3. Anomalies

- **Update Anomaly:** Flight city details must be updated in multiple records.
- **Insertion Anomaly:** A flight cannot be stored without a booking.
- **Deletion Anomaly:** Deleting the last booking may remove flight information.

4. 1NF

The relation is in **1NF** because all attributes contain atomic values and there are no repeating groups.

5. 2NF

The key is (FlightNo, Date, PassID).

Partial dependencies exist:

- PassID → PassName
- FlightNo → DeptCity, ArrCity

Therefore, decompose the relation.

6. 3NF

Remove the partial and transitive dependencies by creating separate tables.

7. BCNF Decomposition

Passenger

Passenger(PassID, PassName)

Flight

Flight(FlightNo, DeptCity, ArrCity)

Booking

Booking(FlightNo, Date, PassID, Seat)

Here, the determinants in each relation are candidate keys.

Final BCNF Schema

Relation	Attributes
Passenger	PassID (PK), PassName
Flight	FlightNo (PK), DeptCity, ArrCity
Booking	FlightNo (PK/FK), Date (PK), PassID (PK/FK), Seat

Conclusion

The airline booking relation is successfully normalized up to **BCNF**. The decomposition reduces redundancy and eliminates insertion, deletion, and update anomalies while maintaining data integrity through primary and foreign keys.

Q5. HR System – Normalize to 3NF

1. Given Case

An HR system for a multinational company stores employee, department, manager, and location details. A suitable relation is:

Employee(EmpID, EmpName, DeptID, DeptName, ManagerID, ManagerName, Location)

2. Functional Dependencies

- EmpID → EmpName, DeptID
- DeptID → DeptName, ManagerID, Location
- ManagerID → ManagerName

Candidate Key: EmpID

3. Anomalies

- **Update Anomaly:** Department or manager details must be changed in several employee records.
- **Insertion Anomaly:** A new department cannot be added without an employee.
- **Deletion Anomaly:** Deleting the last employee of a department may remove department information.

4. 1NF

The relation is in **1NF** because all attributes contain atomic values and there are no repeating groups.

5. 2NF

Since EmpID is a single-attribute key, there are no partial dependencies. Therefore, the relation satisfies **2NF**.

6. Transitive Dependencies

There are transitive dependencies:

EmpID → DeptID → DeptName, ManagerID, Location

and

EmpID → ManagerID → ManagerName

These must be removed to achieve 3NF.

7. 3NF Decomposition

Employee

Employee(EmpID, EmpName, DeptID)

Department

Department(DeptID, DeptName, ManagerID, Location)

Manager

Manager(ManagerID, ManagerName)

Final 3NF Schema

Relation	Attributes
Employee	EmpID (PK), EmpName, DeptID (FK)
Department	DeptID (PK), DeptName, ManagerID (FK), Location
Manager	ManagerID (PK), ManagerName

Conclusion

The HR schema is normalized to **3NF** by removing transitive dependencies. The decomposition reduces redundancy, avoids update, insertion, and deletion anomalies, and maintains data integrity.

Q6. Library System – Identify MVDs and Decompose to 4NF

1. Given Relation

Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

A library system stores member details, book details, and borrowing information in a single relation. When a member borrows multiple books, the same member information may be repeated.

2. Functional Dependencies

The main functional dependencies are:

- MemberID → MemberName
- BookID → Title, Author, Genre
- (MemberID, BookID) → BorrowDate

Here, MemberID identifies a member and BookID identifies a particular book.

3. Multivalued Dependency (MVD)

A **Multivalued Dependency (MVD)** occurs when one attribute determines a set of independent values of another attribute.

It is represented as **A →→ B**.

In this case:

MemberID →→ BookID

This means that one member can be associated with multiple books independently.

4. Problems / Redundancy

Suppose a member borrows several books. The member's name will be repeated for every borrowed book.

This can cause:

- **Update anomaly:** Member information may need to be updated in multiple records.
- **Insertion anomaly:** Member information may be difficult to store without borrowing information.
- **Deletion anomaly:** Deleting a borrowing record may accidentally remove important member information.

5. 1NF

The relation satisfies **1NF** because:

- All attributes contain atomic values.
- There are no repeating groups.
- Each field contains a single value.

6. 2NF and 3NF

The relation can be separated based on the dependencies:

MemberID → MemberName

and

BookID → Title, Author, Genre

The borrowing information depends on the combination of member and book:

(MemberID, BookID) → BorrowDate

7. Need for 4NF

4NF (Fourth Normal Form) deals mainly with **multivalued dependencies**.

A relation is in 4NF when:

- It is already in BCNF.
- It has no non-trivial multivalued dependency where the determinant is not a super key.

Since MemberID →→ BookID creates an independent multivalued relationship, the relation should be decomposed.

8. 4NF Decomposition

Member Relation

Member(MemberID, MemberName)

- **Primary Key:** MemberID

Book Relation

Book(BookID, Title, Author, Genre)

- **Primary Key:** BookID

Borrow Relation

Borrow(MemberID, BookID, BorrowDate)

- **Primary Key:** (MemberID, BookID)
- MemberID → Foreign Key
- BookID → Foreign Key

9. Final 4NF Schema

Relation	Attributes
Member	MemberID (PK), MemberName
Book	BookID (PK), Title, Author, Genre
Borrow	MemberID (PK/FK), BookID (PK/FK), BorrowDate

10. Advantages of Decomposition

- Reduces data redundancy.
- Removes multivalued dependency problems.
- Prevents update anomalies.
- Prevents insertion and deletion anomalies.
- Maintains referential integrity using foreign keys.
- Makes the database easier to maintain.

Conclusion

The given **Library** relation is decomposed into **Member, Book, and Borrow** relations to remove the multivalued dependency and achieve **4NF**. The final design stores member, book, and borrowing information separately, thereby reducing redundancy and improving data consistency and integrity.

Q7. Telecom Billing System – Candidate Keys, Primary Keys and Functional Dependencies

1. Given Relation

A telecom billing system stores customer, plan, call, and billing information. A suitable relation is:

Telecom(CustomerID, CustomerName, PhoneNo, PlanID, PlanName, CallID, CallDuration, CallCharge)

2. Functional Dependencies

The main functional dependencies are:

- CustomerID → CustomerName, PhoneNo, PlanID
- PhoneNo → CustomerID
- PlanID → PlanName
- CallID → CustomerID, CallDuration, CallCharge

3. Candidate Keys

A **candidate key** is a minimal attribute or set of attributes that uniquely identifies every record.

Possible candidate keys are:

- CustomerID
- PhoneNo
- CallID *(if each billing record represents one unique call)*

4. Primary Key

The **primary key** is selected from the candidate keys.

For the customer information, we can choose:

Primary Key = CustomerID

Other unique identifiers can be maintained as candidate/alternate keys where applicable.

5. Dependency Analysis

CustomerID → CustomerName, PhoneNo, PlanID

This means CustomerID determines the customer's name, phone number, and plan.

PlanID → PlanName

This means PlanID determines the plan name.

CallID → CustomerID, CallDuration, CallCharge

This means CallID determines the customer and details of the call.

6. Redundancy and Anomalies

Storing all telecom information in one relation can cause:

- **Update anomaly:** Plan or customer information may need to be updated in many records.
- **Insertion anomaly:** A new plan may not be stored without customer information.
- **Deletion anomaly:** Deleting the last call may remove important customer or plan information.

Conclusion

The telecom billing schema contains several functional dependencies and candidate keys. Identifying these keys and dependencies is the first step toward normalization. Proper normalization reduces redundancy, prevents anomalies, and ensures **data integrity**.

Q8. Retail Inventory System – Complete Normalization

1. Given Relation

Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

The retail inventory system stores product, supplier, category, price, and warehouse information in one relation.

2. Functional Dependencies

The main functional dependencies are:

- PID → PName, SupplierID, Category, Price, Warehouse
- SupplierID → SupplierName

Candidate Key: PID

3. Anomalies

Update Anomaly:
If a supplier's name changes, it must be updated for every product supplied by that supplier.

Insertion Anomaly:
A new supplier cannot be added unless a product is associated with the supplier.

Deletion Anomaly:
If the last product supplied by a supplier is deleted, the supplier's information may also be lost.

4. 1NF

The relation is in **1NF** because:

- All attributes contain atomic values.
- There are no repeating groups.
- Each field contains a single value.

5. 2NF

The candidate key is PID, which is a single attribute. Therefore, there are **no partial dependencies**.

Hence, the relation satisfies **2NF**.

6. 3NF

There is a transitive dependency:

PID → SupplierID → SupplierName

To remove this dependency, separate supplier information from product information.

7. Decomposition

Product

Product(PID, PName, SupplierID, Category, Price, Warehouse)

Supplier

Supplier(SupplierID, SupplierName)

8. BCNF Check

In the **Product** relation:

PID → PName, SupplierID, Category, Price, Warehouse

PID is the candidate key.

In the **Supplier** relation:

SupplierID → SupplierName

SupplierID is the candidate key.

Therefore, both relations satisfy **BCNF** under these assumptions.

Final Normalized Schema

Relation	Attributes
Product	PID (PK), PName, SupplierID (FK), Category, Price, Warehouse
Supplier	SupplierID (PK), SupplierName

Conclusion

The original retail inventory relation is completely normalized by removing the transitive dependency between **SupplierID and SupplierName**. The final design reduces redundancy, prevents update, insertion, and deletion anomalies, and maintains data integrity.

Q9. School ERP – Fully Normalized Design up to BCNF

1. Given Schema

A School ERP system stores student, course, teacher, department, and grade information in one relation:

School(StudentID, StudentName, CourseID, CourseName, TeacherID, TeacherName, DeptID, DeptName, Grade)

Storing all information in one table creates redundancy and may cause insertion, deletion, and update anomalies.

2. Functional Dependencies

The main functional dependencies are:

- StudentID → StudentName
- CourseID → CourseName, TeacherID, DeptID
- TeacherID → TeacherName
- DeptID → DeptName
- (StudentID, CourseID) → Grade

Candidate Key: (StudentID, CourseID)

3. Anomalies

Update Anomaly:
If a teacher's name changes, it must be updated in several records.

Insertion Anomaly:
A new course or teacher cannot be stored easily without an enrolled student.

Deletion Anomaly:
If the last student enrolled in a course is deleted, the course and teacher information may also be lost.

4. 1NF

The relation is in **1NF** because:

- Every attribute contains atomic values.
- There are no repeating groups.
- Each field contains a single value.

5. 2NF

The candidate key is (StudentID, CourseID).

There are partial dependencies:

- StudentID → StudentName
- CourseID → CourseName, TeacherID, DeptID

Therefore, student and course information should be separated.

6. 3NF

There are transitive dependencies:

CourseID → TeacherID → TeacherName

CourseID → DeptID → DeptName

These dependencies cause redundancy. Therefore, Teacher and Department information must be stored separately.

7. BCNF Decomposition

Student Relation

Student(StudentID, StudentName)

Primary Key: StudentID

Course Relation

Course(CourseID, CourseName, TeacherID, DeptID)

Primary Key: CourseID

Foreign Keys: TeacherID, DeptID

Teacher Relation

Teacher(TeacherID, TeacherName)

Primary Key: TeacherID

Department Relation

Department(DeptID, DeptName)

Primary Key: DeptID

Enrollment Relation

Enrollment(StudentID, CourseID, Grade)

Primary Key: (StudentID, CourseID)

Foreign Keys: StudentID, CourseID

8. BCNF Verification

For each relation:

- StudentID → StudentName and StudentID is a key.
- CourseID → CourseName, TeacherID, DeptID and CourseID is a key.
- TeacherID → TeacherName and TeacherID is a key.
- DeptID → DeptName and DeptID is a key.
- (StudentID, CourseID) → Grade and the combination is the key.

Thus, every determinant is a candidate key. Therefore, the decomposed relations satisfy **BCNF**.

9. Final Normalized Schema

Relation	Attributes
Student	StudentID (PK), StudentName
Course	CourseID (PK), CourseName, TeacherID (FK), DeptID (FK)
Teacher	TeacherID (PK), TeacherName
Department	DeptID (PK), DeptName
Enrollment	StudentID (PK/FK), CourseID (PK/FK), Grade

10. Advantages

- Reduces duplicate student, teacher, and department information.
- Eliminates partial dependencies.
- Eliminates transitive dependencies.
- Reduces insertion, deletion, and update anomalies.
- Maintains referential integrity through foreign keys.
- Makes the School ERP database easier to maintain.

Conclusion

The poorly normalized School ERP relation is successfully transformed into a set of relations in **BCNF**. The decomposition removes partial and transitive dependencies, reduces redundancy, eliminates major data anomalies, and ensures better **data consistency and integrity**.

Q10. Movie Streaming Platform – Join Dependencies and 5NF

1. Given Case

A movie streaming platform stores information about movies, actors, and streaming platforms. A movie can have multiple actors and can be available on multiple streaming platforms.

A suitable relation is:

MovieStream(MovieID, ActorID, PlatformID)

2. Functional Dependencies

Assuming there is no single attribute that determines the other attributes, the main relationships are represented through combinations of attributes.

The possible key is:

Candidate Key: (MovieID, ActorID, PlatformID)

The relation may contain several combinations of movies, actors, and platforms, which can result in redundant data.

3. Join Dependency

A **Join Dependency (JD)** specifies that a relation can be reconstructed by joining multiple smaller relations.

It can be represented as:

*(MovieID, ActorID), (MovieID, PlatformID), (ActorID, PlatformID)

This means that the information about movie-actor, movie-platform, and actor-platform relationships can be stored separately.

4. Problems in the Original Relation

When all three relationships are stored in one table, the same information may be repeated.

For example, if one movie has several actors and is available on several platforms, multiple combinations may have to be stored.

This can lead to:

- **Data redundancy**
- **Update anomaly**
- **Insertion anomaly**
- **Deletion anomaly**

- Increased storage requirements

5. 1NF

The relation satisfies **1NF** because:

- All attributes contain atomic values.
- There are no repeating groups.
- Each field contains a single value.

6. 2NF

If the candidate key is (MovieID, ActorID, PlatformID), all relationship information should depend on the appropriate combination of attributes.

If partial dependencies exist, they should be removed through decomposition.

7. 3NF

There should be no transitive dependency between non-key attributes. Since the main attributes represent independent entities and relationships, the relation can be further decomposed to avoid redundancy.

8. BCNF

A relation satisfies **BCNF** when every determinant is a candidate key.

After decomposition, the relationship tables use combinations of their attributes as keys, helping maintain the required dependencies.

9. 5NF

Fifth Normal Form (5NF) is also known as **Project-Join Normal Form (PJ/NF)**.

A relation is in **5NF** when every non-trivial join dependency is implied by the candidate keys.

The purpose of 5NF is to remove redundancy caused by complex **join dependencies** that cannot be removed by 3NF or BCNF.

10. 5NF Decomposition

The original relation can be decomposed into three relations.

MovieActor

MovieActor(MovieID, ActorID)

This stores which actors are associated with each movie.

MoviePlatform

MoviePlatform(MovieID, PlatformID)

This stores which platforms provide each movie.

ActorPlatform

ActorPlatform(ActorID, PlatformID)

This stores the relationship between actors and platforms, according to the assumed business rule.

11. Final 5NF Schema

Relation	Attributes
MovieActor	MovieID (PK/FK), ActorID (PK/FK)
MoviePlatform	MovieID (PK/FK), PlatformID (PK/FK)
ActorPlatform	ActorID (PK/FK), PlatformID (PK/FK)

Each relation contains a composite primary key consisting of its two attributes.

12. Lossless-Join Verification

A decomposition is **lossless** if joining the decomposed relations produces the original valid information without losing tuples or introducing incorrect tuples.

The original relation can be reconstructed using:

MovieActor ⋈ MoviePlatform ⋈ ActorPlatform

If the resulting relation contains exactly the valid movie-actor-platform combinations, the decomposition is **lossless**.

Thus, the decomposition preserves the required information.

13. Advantages of 5NF Decomposition

- Removes complex join dependency redundancy.
- Reduces unnecessary duplicate records.
- Prevents update anomalies.
- Prevents insertion and deletion anomalies.
- Improves database consistency.
- Makes relationships easier to manage.
- Provides a more efficient relational design.

Conclusion

The movie streaming relation contains a complex relationship between **movies, actors, and streaming platforms**. By identifying the join dependency and decomposing the relation into **MovieActor, MoviePlatform, and ActorPlatform**, the database can be represented in **5NF**. The decomposition reduces redundancy and, under the stated join-dependency assumption, provides a **lossless-join design** with improved data integrity.